ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ Р.МОЛДОВЫ
ФАКУЛЬТЕТ МАТЕМАТИКИ И ИНФОРМАТИКИ
ДЕПАРТАМЕНТ ИНФОРМАТИКИ

Дулева Анастасия

Отчет

по дисциплине „PYTHON для приложений”

Руководитель: Плешка Наталья, лектор

Автор: Дулева Анастасия

Кишинев, 2026

1. Краткая постановка задачи 2
2. Логика реализованных алгоритмов 2
3. Скрины интерфейса и их назначение 5
4. Структуры данных, использованные в приложении 6
5. Обоснование использования Pandas и Flask 7
6. Описание функционала с кодом и пояснениями 7
Выводы 11

Вариант 2

1. Краткая постановка задачи

Целью работы является разработка веб-приложения на Flask для анализа данных интернет-магазина, хранящихся в CSV-файле. Приложение должно автоматически загружать данные, обрабатывать их с помощью библиотеки Pandas и отображать результаты анализа на веб-странице.

В рамках задания необходимо вычислить основные показатели продаж, включая общий доход, количество заказов, наиболее продаваемый товар, категорию с максимальными продажами и город с наибольшим числом заказов. Также требуется реализовать таблицу заказов, сводную таблицу по категориям, фильтрацию данных по городам и построение графика продаж.

2. Логика реализованных алгоритмов

Работа приложения выполняется по следующей последовательности:

1. Проверяется существование CSV-файла с данными.

```
if not os.path.exists(FILE_PATH):
        return "CSV файл не найден!"
```

   На этом этапе программа проверяет, существует ли файл с данными. Если файл отсутствует - выполнение останавливается и выводится сообщение об ошибке.

2. Загружаются данные с помощью Pandas.

```
df = pd.read_csv(FILE_PATH)
```

   CSV-файл загружается в DataFrame, который используется для дальнейшего анализа данных.

3. Удаляются пропущенные значения.

```
df = df.dropna()
```

   Все строки с отсутствующими значениями удаляются для обеспечения корректности вычислений.

4. Вычисляется новый столбец общей стоимости заказа:

```
df['Total_Price'] = df['Quantity'] * df['Unit Price']
```

   Для каждого заказа рассчитывается общая стоимость по формуле: количество * цена за единицу товара.

5. Вычисляются основные показатели:

```
total_orders = len(df)

    total_revenue = df['Total_Price'].sum()

    best_selling_product = (
        df.groupby('Product Name')['Quantity']
        .sum()
        .idxmax()
    )

    best_category = (
        df.groupby('Category')['Total_Price']
        .sum()
        .idxmax()
    )

    top_city = (
        df['City']
        .value_counts()
        .idxmax()
    )
```

a) общее количество заказов;

b) общий доход;

c) самый продаваемый товар;

d) категория с максимальными продажами;

e) город с наибольшим числом заказов.

6. Выполняется фильтрация данных по выбранному городу.

```
selected_city = request.args.get('city')

    if selected_city and selected_city != 'All':
        filtered_df = df[df['City'] == selected_city]
    else:
        filtered_df = df
```

Пользователь может выбрать город через HTML-форму.

Если выбран конкретный город - данные фильтруются, иначе отображаются все записи.

7. Формируется сводная таблица по категориям.

```
category_summary = (
        df.groupby('Category')['Total_Price']
        .sum()
        .reset_index()
    )
```

Данные группируются по категориям товаров и суммируются для анализа продаж.

8. Строится график продаж.

```
plt.figure(figsize=(8, 5))
    plt.bar(
        category_summary['Category'],
        category_summary['Total_Price']
    )

    plt.xticks(rotation=45)
    plt.title('Sales by Category')
    plt.tight_layout()

    chart_path = 'static/chart.png'

    plt.savefig(chart_path)
    plt.close()
```

Строится столбчатый график продаж по категориям товаров. График сохраняется как изображение и отображается на сайте.

9. Данные передаются в HTML-шаблон Flask.

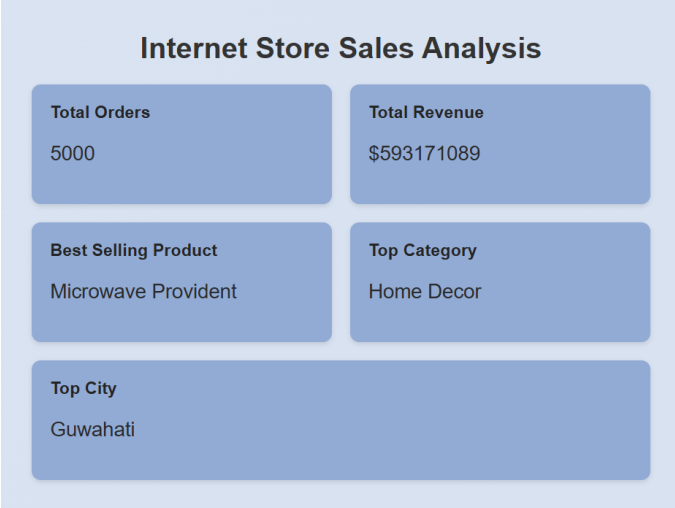
```
return render_template(
        'index.html',
        total_orders=total_orders,
        total_revenue=round(total_revenue, 2),
        best_selling_product=best_selling_product,
        best_category=best_category,
        top_city=top_city,
        orders=orders,
        summary=summary,
```

```
        cities=cities,
        selected_city=selected_city
    )
```

Все рассчитанные данные передаются в HTML-шаблон и отображаются пользователю в браузере.

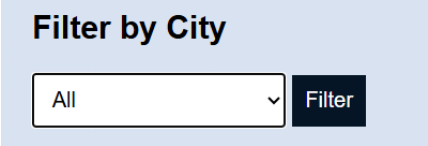
3. Скрины интерфейса и их назначение

1. Главная страница приложения



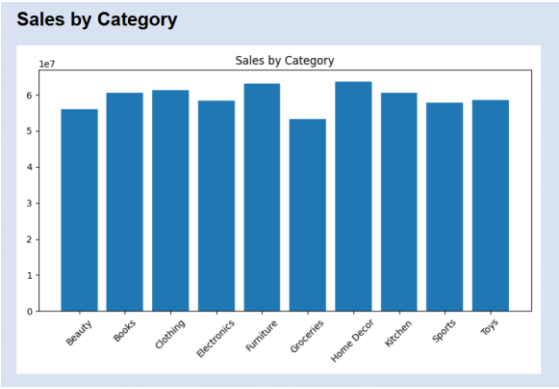
Назначение: отображение общей статистики (заказы, доход, лучший товар, категория, город).

2. Фильтр по городам



Назначение: позволяет пользователю выбрать конкретный город для анализа данных.

3. График продаж по категориям



Назначение: визуализация распределения дохода по категориям товаров.

4. Таблица заказов

Orders Table

Order ID	Order Date	Customer Name	Region	City	Category	Sub-Category	Product Name	Quantity	Unit Price	Discount	Sales	Profit	Payment Mode
10001	10/19/2024	Kashvi Varty	South	Bangalore	Books	Non-Fiction	Non-Fiction Ipsum	2	36294	5	68958.6	10525.09	Debit Card
10002	8/30/2025	Advik Desai	North	Delhi	Groceries	Rice	Rice Nemo	1	42165	20	33732.0	6299.66	Debit Card
10003	11/4/2023	Rhea Kalla	East	Patna	Kitchen	Juicer	Juicer Odio	4	64876	20	207603.2	19850.27	Credit Card
10004	5/23/2025	Anika Sen	East	Kolkata	Groceries	Oil	Oil Doloribus	5	37320	15	158610.0	36311.02	UPI
10005	1/19/2025	Akarsh Kaul	West	Pune	Clothing	Kids Wear	Kids Wear Quo	1	50037	10	45033.3	9050.04	Debit Card
10006	4/6/2025	Vardaniya Jayaraman	West	Pune	Furniture	Chair	Chair Assumenda	5	40287	15	171219.75	23722.84	Credit Card
10007	10/13/2023	Drishya Khare	West	Mumbai	Clothing	Accessories	Accessories Minima	2	3636	5	6908.4	680.26	Debit Card
10008	8/17/2025	Misha Dua	North	Lucknow	Books	Biography	Biography Vel	1	15885	10	14296.5	1289.03	Debit Card
10009	3/7/2025	Arhaan Vala	North	Jaipur	Groceries	Spices	Spices Expedita	1	40834	20	32667.2	3700.89	Credit Card
10010	12/2/2024	Lavanya Hayer	West	Goa	Kitchen	Juicer	Juicer Voluptatibus	2	16686	15	28366.2	5703.09	UPI

Назначение: отображение всех заказов с рассчитанной стоимостью.

5. Сводная таблица

Category Summary

Category	Total Sales
Beauty	55936828
Books	60587154
Clothing	61238855
Electronics	58317668
Furniture	63160552
Groceries	53192552
Home Decor	63692686
Kitchen	60560739
Sports	57857786
Toys	58626269

Назначение: показ суммарных продаж по категориям товаров.

4. Структуры данных, использованные в приложении

В проекте использовались следующие структуры данных:

DataFrame (Pandas) - основная структура для хранения и обработки данных из CSV-файла.

Словари (dict) - для передачи данных в HTML-шаблон через Flask (to_dict(orient='records')).

Списки (list) - для хранения уникальных городов и формирования выпадающего списка.

Агрегированные таблицы Pandas - результат группировки данных (groupby).

Эти структуры позволяют эффективно выполнять анализ и визуализацию данных.

5. Обоснование использования Pandas и Flask

Pandas используется, так как предоставляет удобные инструменты для:

- обработки CSV-файлов;
- фильтрации и очистки данных;
- группировки и агрегации;
- выполнения вычислений над таблицами.

Flask используется для:

- создания веб-приложения;
- обработки HTTP-запросов;
- передачи данных в HTML-шаблоны;
- взаимодействия пользователя с системой через браузер.

Совместное использование Pandas и Flask позволяет создать полноценное аналитическое веб-приложение.

6. Описание функционала с кодом и пояснениями

6.1 Загрузка и проверка файла

```
if not os.path.exists(FILE_PATH):
        return "CSV файл не найден!"
```

На первом этапе приложение проверяет, существует ли CSV-файл по указанному пути. Это выполняется с помощью функции os.path.exists(), которая возвращает True, если файл найден, и False в противном случае. Если файл отсутствует, выполнение программы прекращается, и пользователю выводится сообщение об ошибке, что позволяет избежать дальнейших сбоев.

```
#чтение CSV
    df = pd.read_csv(FILE_PATH)


    #удаление пропущенных значений
    df = df.dropna()
```

После успешной проверки файл загружается в DataFrame с помощью pd.read_csv(). Далее выполняется очистка данных методом dropna(), который удаляет все строки с пропущенными значениями, обеспечивая корректность дальнейших вычислений.

6.2 Расчёт общей стоимости заказа

```
df['Total_Price'] = df['Quantity'] * df['Unit Price']
```

На этом этапе создаётся новый столбец Total_Price, который рассчитывается как произведение количества товаров на их цену за единицу. Таким образом формируется общая стоимость каждого отдельного заказа, что позволяет в дальнейшем анализировать доход.

6.3 Основные показатели

```
total_orders = len(df)


    total_revenue = df['Total_Price'].sum()
```

Общее количество заказов определяется как число строк в таблице с помощью функции len(). Общий доход рассчитывается путём суммирования всех значений столбца Total_Price с использованием метода sum(), что позволяет получить итоговую выручку магазина.

6.4 Самые важные метрики

```
# самый продаваемый товар
    best_selling_product = (
        df.groupby('Product Name')['Quantity']
        .sum()
        .idxmax()
    )

    #категория с наибольшими продажами
    best_category = (
        df.groupby('Category')['Total_Price']
        .sum()
        .idxmax()
    )

    #Город с наибольшим количеством заказов
    top_city = (
        df['City']
        .value_counts()
        .idxmax()
    )
```

Для определения самого продаваемого товара данные группируются по названию продукта, после чего суммируется количество проданных единиц. Метод idxmax() возвращает название товара с максимальным значением. Аналогично определяется категория с наибольшими продажами, где суммируется общий доход по каждой категории. Для определения города с наибольшим количеством заказов используется value_counts(), который подсчитывает количество заказов по каждому городу, а затем выбирается максимальное значение.

6.5 Фильтрация по городу

```
selected_city = request.args.get('city')


    if selected_city and selected_city != 'All':
        filtered_df = df[df['City'] == selected_city]
    else:
        filtered_df = df
```

На этом этапе реализуется фильтрация данных. Значение выбранного города передаётся через GET-запрос из HTML-формы. Если пользователь выбрал конкретный город, выполняется фильтрация DataFrame по этому значению. Если выбрано значение “All”, отображаются все данные без изменений.

6.6 Сводная таблица

```
category_summary = (
        df.groupby('Category')['Total_Price']
        .sum()
        .reset_index()
    )
```

Сводная таблица формируется путём группировки данных по категориям товаров. Для каждой категории суммируется общая стоимость продаж. Метод reset_index() используется для преобразования результата обратно в обычную таблицу, удобную для отображения на веб-странице.

6.7 Построение графика

```
plt.figure(figsize=(8, 5))
    plt.bar(
        category_summary['Category'],
```

```
        category_summary['Total_Price']
    )

    plt.xticks(rotation=45)
    plt.title('Sales by Category')
    plt.tight_layout()

    chart_path = 'static/chart.png'

    plt.savefig(chart_path)
    plt.close()
```

На этом этапе создаётся визуализация данных с помощью библиотеки Matplotlib. Формируется столбчатая диаграмма, где по оси X отображаются категории товаров, а по оси Y - сумма продаж. Подписи категорий поворачиваются на 45 градусов для лучшей читаемости. После настройки внешний вид графика сохраняется в файл chart.png, который затем отображается на веб-странице. После сохранения график закрывается для освобождения памяти.

6.8 Передача данных в HTML

```
return render_template(
        'index.html',
        total_orders=total_orders,
        total_revenue=round(total_revenue, 2),
        best_selling_product=best_selling_product,
        best_category=best_category,
        top_city=top_city,
        orders=orders,
        summary=summary,
        cities=cities,
        selected_city=selected_city
    )
```

На последнем этапе все рассчитанные данные передаются в HTML-шаблон с помощью функции render_template(). Это позволяет динамически отображать результаты анализа на веб-странице. Передаются как числовые показатели (заказы, доход), так и структурированные данные (таблицы и списки), а также выбранный пользователем город для сохранения состояния фильтра.

Выводы

В ходе выполнения данной работы было разработано полноценное веб-приложение для анализа данных интернет-магазина с использованием языка программирования Python, библиотеки Pandas и веб-фреймворка Flask. Основной задачей проекта являлась автоматизация обработки данных о заказах, их анализ и визуализация результатов в удобном для пользователя виде через веб-интерфейс.

В процессе работы приложение успешно выполняет загрузку данных из CSV-файла, проверяет его наличие, а также обрабатывает возможные пропуски в данных, что обеспечивает корректность дальнейших вычислений. С помощью библиотеки Pandas были реализованы основные операции анализа данных, включая вычисление общей стоимости каждого заказа, определение общего количества заказов, общего дохода, а также выявление наиболее популярных товаров, категорий и городов.

Дополнительно в приложении была реализована группировка данных по категориям товаров, что позволило сформировать сводную таблицу продаж. На её основе был построен график, визуально отображающий распределение дохода по категориям, что облегчает восприятие аналитической информации. Также была реализована фильтрация данных по городам, что позволяет пользователю анализировать заказы в разрезе конкретного региона.

Использование Flask позволило создать удобный веб-интерфейс, через который пользователь может взаимодействовать с системой. Данные динамически передаются в HTML-шаблон и отображаются в виде таблиц, карточек со статистикой и графика. Это делает приложение наглядным и удобным для анализа информации без необходимости работы с кодом.

Таким образом, в ходе выполнения работы были закреплены практические навыки работы с библиотекой Pandas для анализа данных и с Flask для создания веб-приложений. Также были отработаны навыки обработки табличных данных, их визуализации и интеграции аналитики в веб-среду.

Полученный результат демонстрирует возможность создания простых, но функциональных аналитических систем, которые могут использоваться для первичного анализа данных в реальных задачах.

Список используемой литературы

1. **Python pentru aplicații/ Python для приложений - https://elearning.usm.md/course/view.php?id=6768**
2. **Документация Pandas - https://pandas.pydata.org/docs/**
3. **Документация Flask - https://flask.palletsprojects.com/en/stable/**